Ingineria Programării

Cursul 07 – 29 martie

1936

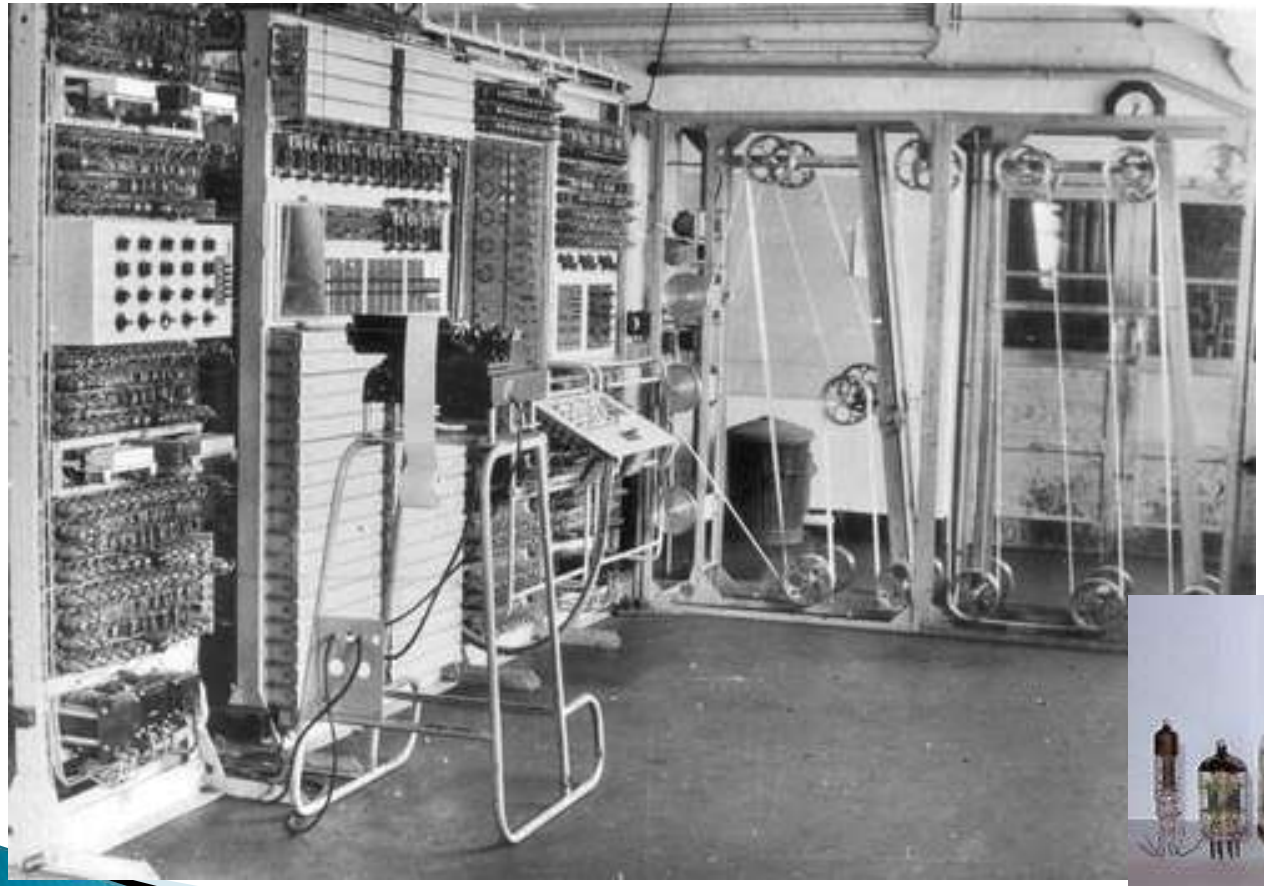
- Alan Turing
  - 'On computable numbers, with an application to the Entscheidungsproblem‘
- First code we could recognize
  - Binary
  - Ran on electronic machines
  - Uses variables, macros, subroutines

1939

- Bletchley Park – The Bombe
- Used to crack Enigma Code
- Created using electro-mechanical relays
- Clock rate ~ 100 ms

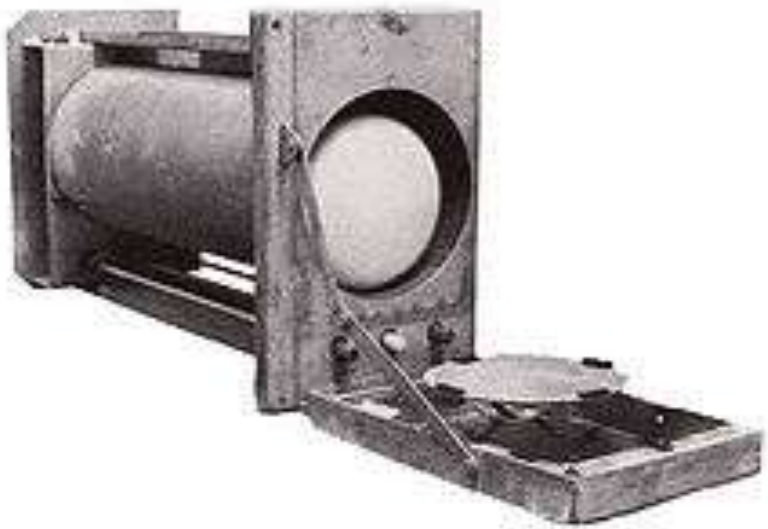
1942

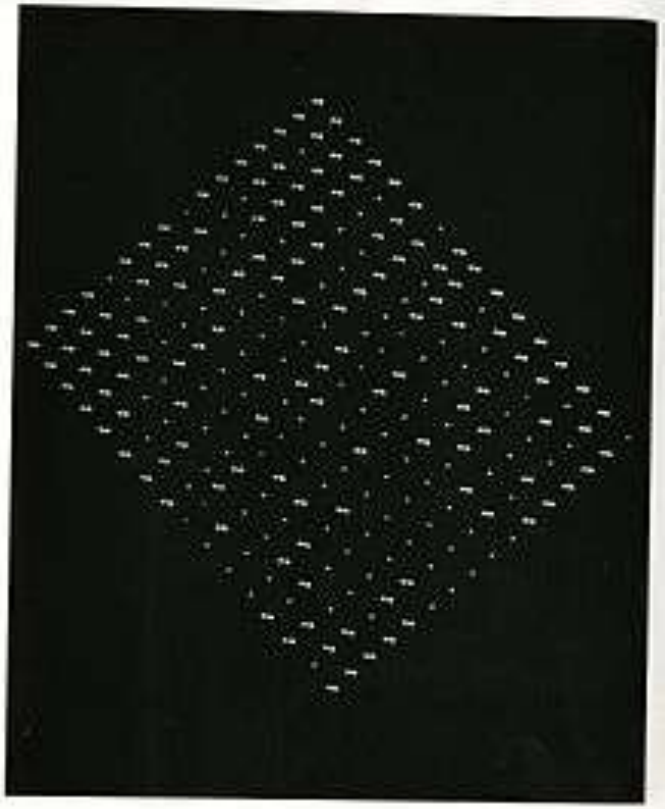
- Colossus- first programmable computer



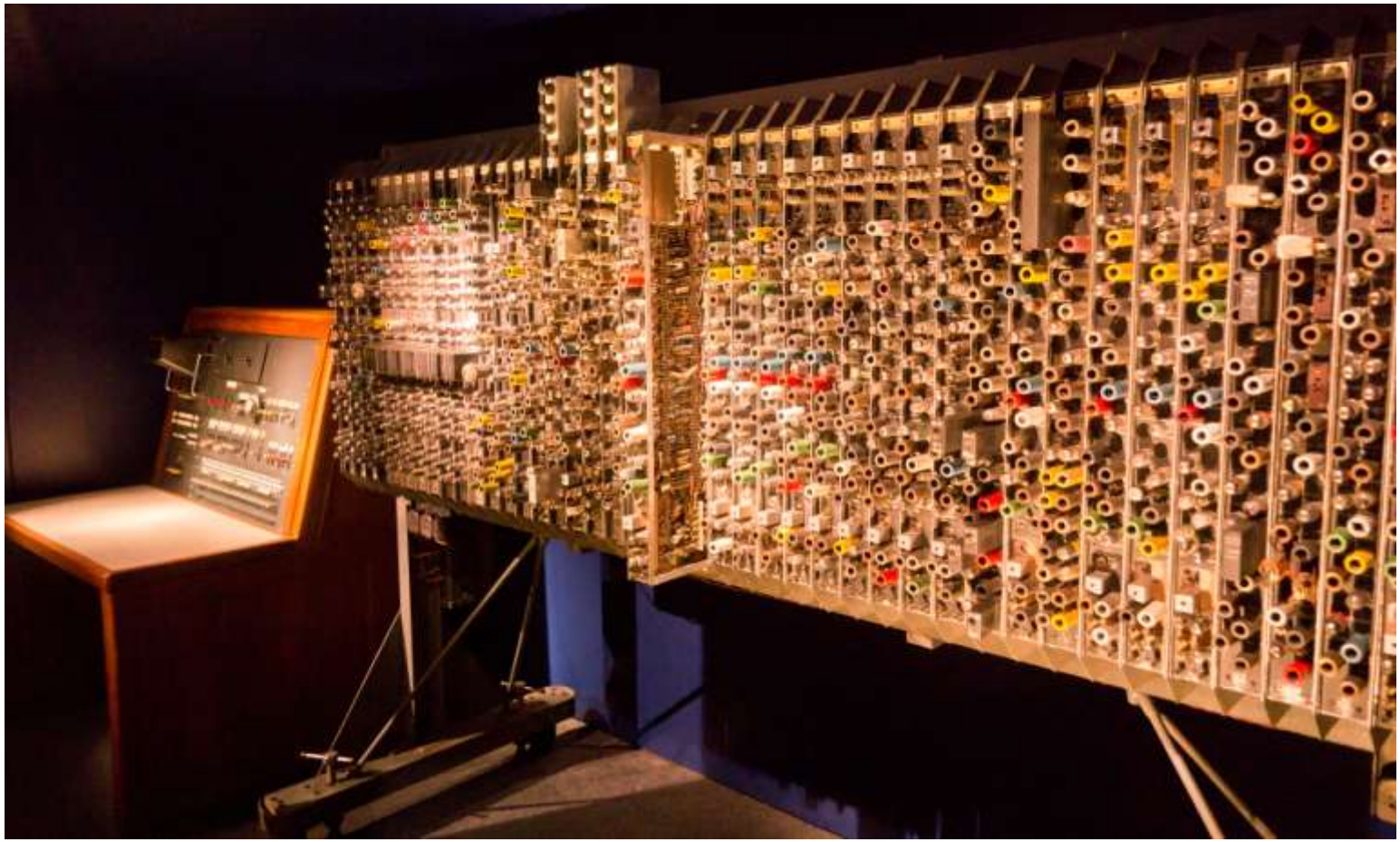
1945 – Design of ACE

- Designed by Alan Turing
- Worked in base 32
- Used Cathode Ray Tubes for memory





1950 – ACE



1945 – What we have so far

- Turing writes code
  - Uses binary base 32
  - Uses add and not
  - Floating point
  - Subroutines
  - Stack (for data and subroutine return addresses)
  - and many more

1945 – What we have so far

- “We shall need a great number of mathematicians of ability”, “there will probably be a good deal of this work to be done” – Turing, *Lecture on the ACE*
- “One of our difficulties will be the maintenance of an appropriate discipline, so that we do not lose track of what we a re doing” – Turing, *Lecture on the ACE*

1945 – What We Have so Far

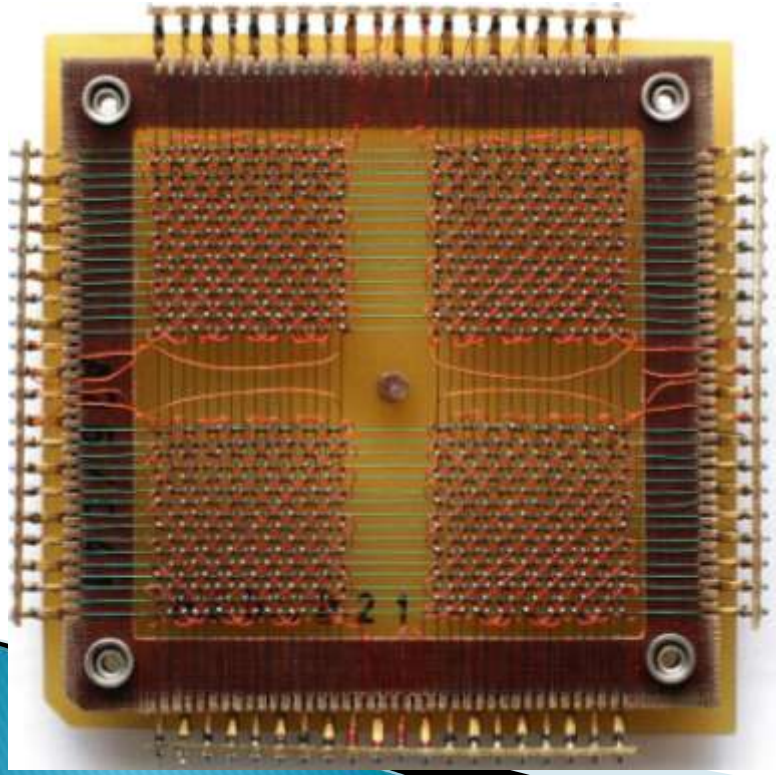
- Computers in the world: O(1)
- Programmers in the world: O(1)

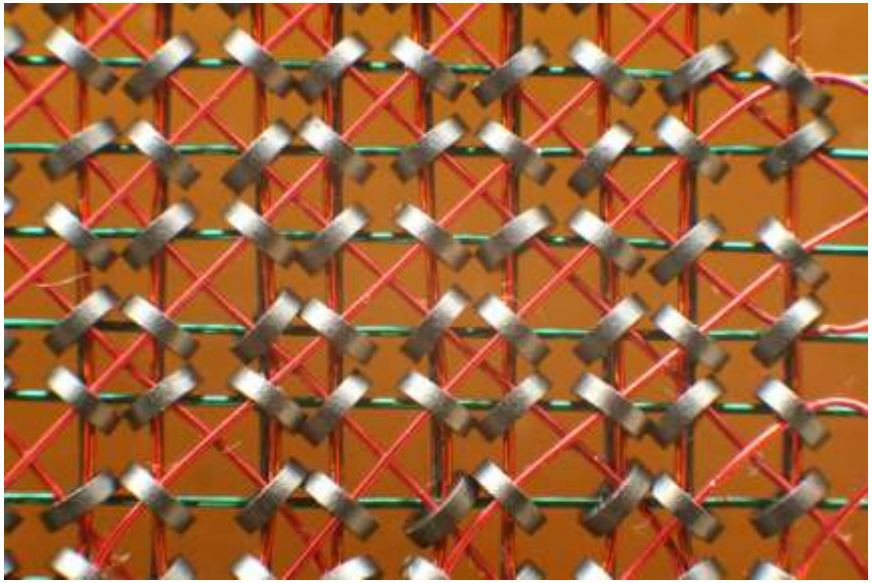
1950 – ACE

- One of the first working computers
  - Vacuum tubes for processing
  - 12 Mercury Delay Lines for memory (audio)
  - 32 instructions/words of 32 bits per line
  - Frequency ~ 1Mhz

1950s–The Memory Problem

- Memory is expensive
  - 1–10 $ per **BIT**
- Cheaper versions: Magnetic-core memory





1953 – Fortran

```
C AREA OF A TRIANGLE WITH A STANDARD SQUARE ROOT FUNCTION
C INPUT - TAPE READER UNIT 5, INTEGER INPUT
C OUTPUT - LINE PRINTER UNIT 6, REAL OUTPUT
C INPUT ERROR DISPLAY ERROR OUTPUT CODE 1 IN JOB CONTROL LISTING
      READ INPUT TAPE 5, 501, IA, IB, IC
  501 FORMAT (3I5)
C IA, IB, AND IC MAY NOT BE NEGATIVE OR ZERO
C FURTHERMORE, THE SUM OF TWO SIDES OF A TRIANGLE
C MUST BE GREATER THAN THE THIRD SIDE, SO WE CHECK FOR THAT, TOO
      IF (IA) 777, 777, 701
  701 IF (IB) 777, 777, 702
  702 IF (IC) 777, 777, 703
  703 IF (IA+IB-IC) 777, 777, 704
  704 IF (IA+IC-IB) 777, 777, 705
  705 IF (IB+IC-IA) 777, 777, 799
  777 STOP 1
C USING HERON'S FORMULA WE CALCULATE THE
C AREA OF THE TRIANGLE
  799 S = FLOATF (IA + IB + IC) / 2.0
      AREA = SQRTF( S * (S - FLOATF(IA)) * (S - FLOATF(IB)) *
     +     (S - FLOATF(IC)))
      WRITE OUTPUT TAPE 6, 601, IA, IB, IC, AREA
  601 FORMAT (4H A= ,I5,5H  B= ,I5,5H  C= ,I5,8H  AREA= ,F10.2,
     +        13H SQUARE UNITS)
      STOP
      END
```

1958 – Functional Programming

- John McCarthy – LISP
  - (defun factorial (n)
  - (if (= n 0) 1
  - (* n (factorial (- n 1)))))

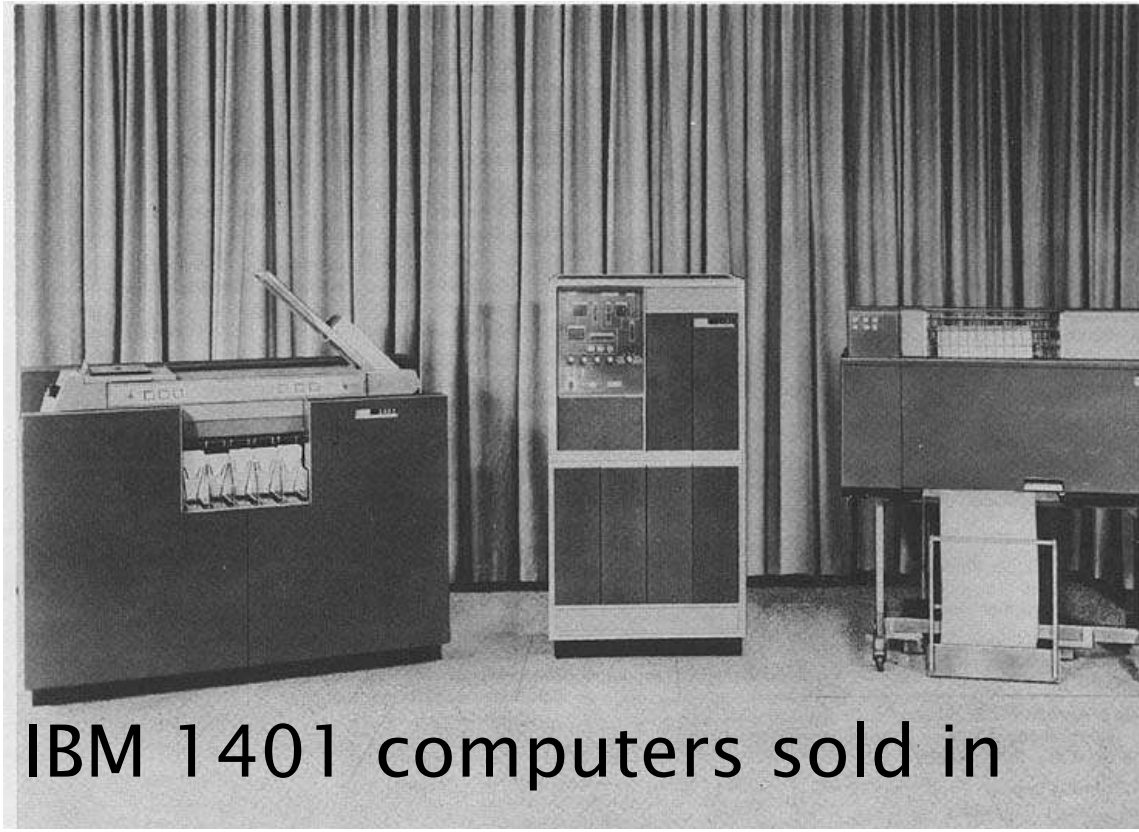
1950s

- In this decade, IBM sold 140 computers
  - Fortran
  - LISP

1960 – What We Have so Far

- Computers in the world: O(1e2)
- Programmers in the world: O(1e3)
- Programmers were:
  - Mathematicians
  - Engineers
  - Scientists
  - Etc.

The Great Invention – The Transistor



- More than 10000 IBM 1401 computers sold in the 1960s
- Decimal Machine

1965

- Computers in the world: O(1e4)
- Programmers in the world: O(1e5)

- In less than 20 years, the number of programmers increased by a factor of 100000
- Though most were not mathematicians, they were experienced, disciplined professionals

1966

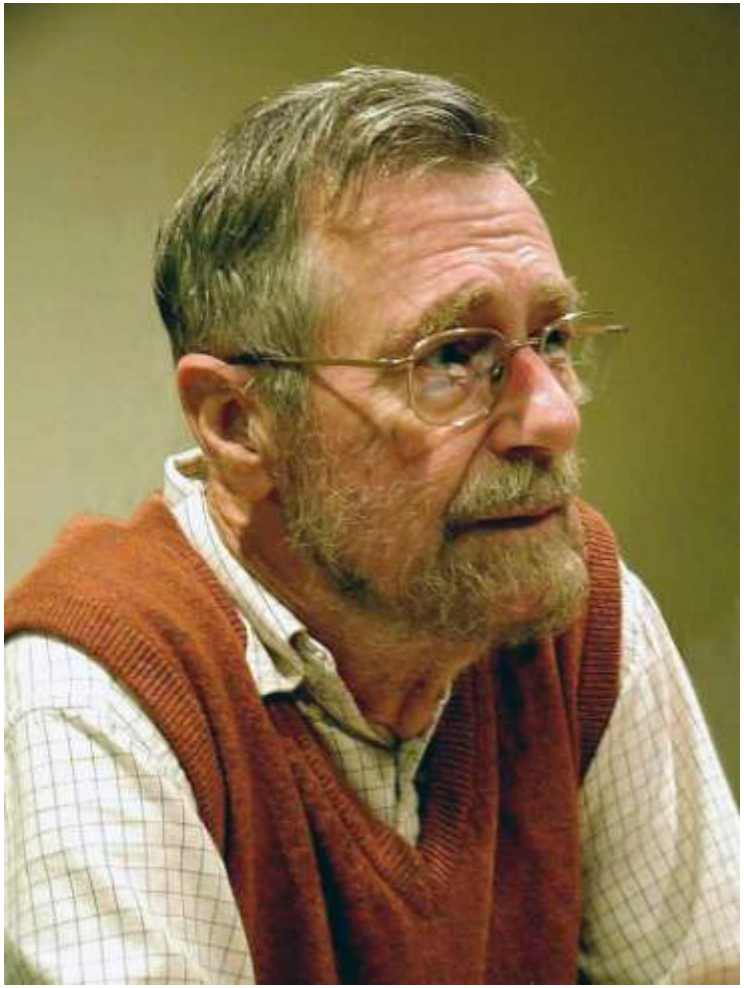
- IBM produces 1000 IBM 360 each month

1966 – Simula 67

- First object oriented programming language
- Created by Ole-Johan Dahl, Kristen Nygard
  - Objects
  - Classes
  - Inheritance
  - Virtual procedures
  - Garbage collection
  - etc.

1968 – GOTO

- Edsger Dijkstra (physicist) – GOTO Considered Harmful
- Beginning of structured programming



1968 – C

- Ken Thompson, Dennis Ritchie, mathematicians

1970

- Computers in the world: O(1e5)
- Programmers in the world: O(1e6)

- Tens of thousands of CS and EE graduates (among the first series)
  - All were young
  - Almost all male

1970s

- Until now
  - Programmers were disciplined
  - Didn't need much management
  - Could manage their own time, work in groups and communicate
  - They understood deadlines
- They used a form of Agile development
  - Agile is the process used by disciplined professionals in the wild

1970 – The Waterfall

- Young, inexperienced programmers, need top down supervision
- Average age of programmers drops by 10 years
- Winston Royce describes the waterfall development model
  - an example of a **flawed**, **non-working** model

1970–1995

- Number of programmers increases exponentially (roughly doubles every 5 years)
- Tens of millions of programmers
- Half of them have less than 5 years experience

What Changed

- Hardware
- Demographichs

What Hasn't Changed

- Software

1995

- Original programmers start retiring
- First generation of career programmers are now about 40 years old
- These career programmers foresee the need for change
  - Scrum – Schwaber, Beedle, DeVos, et al.
  - XP – Beck, Cunningham
  - FDD – Coad
  - Crystal – Cockburn

2001–Agile Manifesto

Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

Agile Requires Discipline

- Working with fixed time spans
- Customer communication
- Collaboration
- Estimating in relative units
- Continuous integration
- etc.

- Agile is NOT a process
- It is a set of promises

Extreme Programming

- The most technical
  - TDD
  - Acceptance tests
  - Simple design
  - Refactoring

Agile

- Turing mentioned discipline
- Agile is about
  - Discipline
  - Craftsmanship
  - Professionalism
- Business understands discipline but does not understand computer programmers or our disciplines

Software craftsmanship

- Software craftsmanship is an approach to software development that emphasizes the coding skills of the software developers themselves.
- A response to the problems of the mainstream software industry, including financial concerns over developer accountability.
- A previously rigorous approach becomes increasingly engineering in nature

Bibliography

- All images and examples courtesy of Robert Cecil Martin and Wikimedia Commons
- Robert C. Martin, Engineering Notebook columns for The C++ Report
- Robert C. Martin, The Future of Programming